kiln: a small Linux sandbox runtime, built layer by layer

A design paper

Stephen Estrada

github.com/stcrestrada/kiln

October 2026

Abstract

kiln is a container runtime of about 2,600 lines of Rust that runs one command at a time in an isolated Linux sandbox. It was built to learn, from the kernel interfaces up, what a container actually is: seven namespaces, an overlay root filesystem swapped in with `pivot_root`, cgroup v2 limits on memory, CPU, processes and block I/O, XFS project quotas for disk, a user namespace that makes sandbox root an unprivileged host UID, a seccomp filter and a Landlock ruleset, a per-sandbox network namespace on a bridge with an nftables egress policy, a policy-enforcing DNS forwarder, and a stateless Prometheus exporter. Each layer was written directly against the kernel's files and system calls (and the standard `ip` and `nft` tools for networking), with no container libraries. This paper describes each layer as a concept and the design decision behind it, what an independent review found and what it taught, and what is still missing.

1 Motivation and goals

kiln is a learning project with a specific aim: real depth in Linux systems concepts (cgroups, namespaces, filesystems, networking, observability). The fastest way to get that depth is to build the thing that exercises it, so kiln re-implements the core of a container runtime: the part that `runc` and Docker hide. Its intended use is running tool calls for AI agents (shell commands, scripts, package installs) where the code is untrusted and its resource use unpredictable.

Goals. (1) Every isolation and limit mechanism is implemented by kiln itself against the kernel interface: `clone`, `mount`, cgroupfs files, `ioctl`s, raw BPF, `nft` rules. (2) Every limit has a measurable failure mode that was triggered on purpose (an OOM kill, a fork bomb, a full disk, a rejected packet). (3) No daemon and no database: all state lives in the kernel and a few directories, so the system can be inspected with `cat`, `nft list` and `findmnt`. (4) Defaults are closed: no network unless asked, limits on by default.

Non-goals (for now). OCI image and runtime-spec compatibility, rootless operation, multi-tenant production hardening, and long-lived sandboxes (the next phase, §15).

Environment. macOS has no namespaces or cgroups, so kiln runs in an Ubuntu 24.04 arm64 VM under Lima (4 vCPU, 4 GiB) with the source tree shared into the VM. The image is Alpine's minirootfs, fetched and checksum-verified by a script.

2 Architecture overview

kiln is one binary with four subcommands. `kiln run` is the runtime: it creates a sandbox, supervises it, and tears it down. `kiln dns` and `kiln metrics` are long-running host services under systemd: a DNS forwarder for sandboxes and a Prometheus exporter. `kiln stats` prints a `docker stats`-style table. Figure 1 shows how they relate.

No daemon, no database. A run's identity is its *run ID*, the start time in nanoseconds (hex) plus kiln's PID: unique, sortable, and used to name both its cgroup (`/sys/fs/cgroup/kiln/<id>`) and its run directory (`/var/lib/kiln/runs/<id>`). An IP lease is a file named after the address whose contents are the run ID, which is how the exporter attributes traffic. The DNS forwarder's per-sandbox policy is a file named after the sandbox's IP. Any tool, including the exporter, finds running sandboxes by listing `/sys/fs/cgroup/kiln/`. The cost of this design is that nothing reconciles state after a crash; §15 returns to this.

Resource ownership through RAII. The supervisor holds a `Cgroup` value and an optional `Network` value. Their destructors run in reverse order on every exit path: the network's nftables element and chain, veth and lease go first, then `cgroup.kill` and `rmdir` of the cgroup, and finally the run directory (unless `--keep`). The overlay mount needs no cleanup: it exists only in the sandbox's mount namespace and disappears with it.

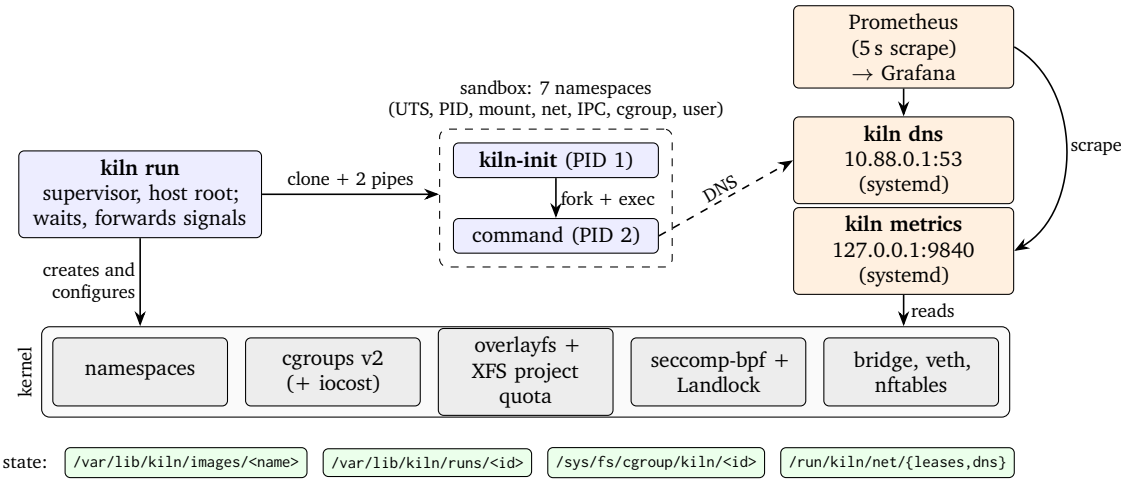


Figure 1: kiln's parts. The supervisor creates each sandbox and configures the kernel on its behalf; the two services run under systemd and are shared by all sandboxes. All state is kernel objects plus the four directories at the bottom.

3 Process and namespace setup

A namespace gives a process its own instance of one kind of global kernel resource. kiln creates five at `clone` time: UTS (hostname), PID (the child becomes PID 1 of a new numbering), mount, network, and IPC (System V shared memory, semaphores and message queues). The child later unshares two more: a cgroup namespace and, last of all, a user namespace. The child has its own 1 MiB stack and no `CLONE_VM`, so it gets a copy of the parent's address space and can read the configuration without sharing memory with the parent.

Figure 2 is the whole startup protocol. Three ordering constraints shape it:

1. **Limits before the first instruction.** The parent moves the child into its cgroup and plugs in its network *before* releasing it. If the child moved itself, the command could start before its limits applied. (`clone3` with `CLONE_INTO_CGROUP` does this in one call; kiln uses the explicit handshake, which it needs anyway for network setup and the ID maps.)
2. **The cgroup namespace after the move.** A cgroup namespace makes the process's *current* cgroup appear as the root. Created before the move, the host's cgroup would become the sandbox's root.
3. **The user namespace last.** Every namespace is owned by the user namespace that was current when it was created, and capabilities only act on namespaces that one's user namespace owns. Creating the user namespace after all the others leaves them owned by the host, so sandbox root, with a full capability set, cannot mount, unmount, change the hostname or reconfigure its network.

File descriptors are the first thing closed. Namespaces, `pivot_root` and Landlock all check *paths* at open time; none of them revokes a descriptor that is already open. A descriptor inherited from whatever started kiln is therefore a handle straight through every wall (§14). The child closes everything except stdio and its two pipe ends before doing anything else, which also means a parent that dies mid-setup shows up as EOF on the pipe.

4 Root filesystem: overlayfs and pivot_root

Every sandbox shares one read-only image and gets a private writable layer (Figure 3). overlayfs merges them: lookups check the upper layer first; the first write to a lower file copies it up; a deletion leaves a *whiteout* (a 0/0 character device) in the upper layer. Two consequences matter for agents. Destroying the image from inside is impossible (`rm -rf /bin` only writes whiteouts), and `upper/` is a precise, free audit log of the run's changes; `--keep` preserves it for inspection. Mount points that kiln itself needs (`/sys/fs/cgroup`, `/workspace`) are created in the image at fetch time, so `upper/` holds only what the sandbox wrote.

Swapping roots. `chroot` only changes how paths are resolved and leaves the old root reachable to a sufficiently privileged process. `pivot_root` changes the mount namespace's root mount itself. kiln uses the `pivot_root(".", ".")` idiom: with the current directory at the new root, the old root ends up stacked on top of it, and `umount2(".", MNT_DETACH)` then removes it, leaving no mount through which the host filesystem can be reached.

Mount propagation. On systemd hosts `/` is a *shared* mount, so even in a new mount namespace, new mounts would propagate back to the host, and `pivot_root` refuses to operate on shared mounts. The child first marks

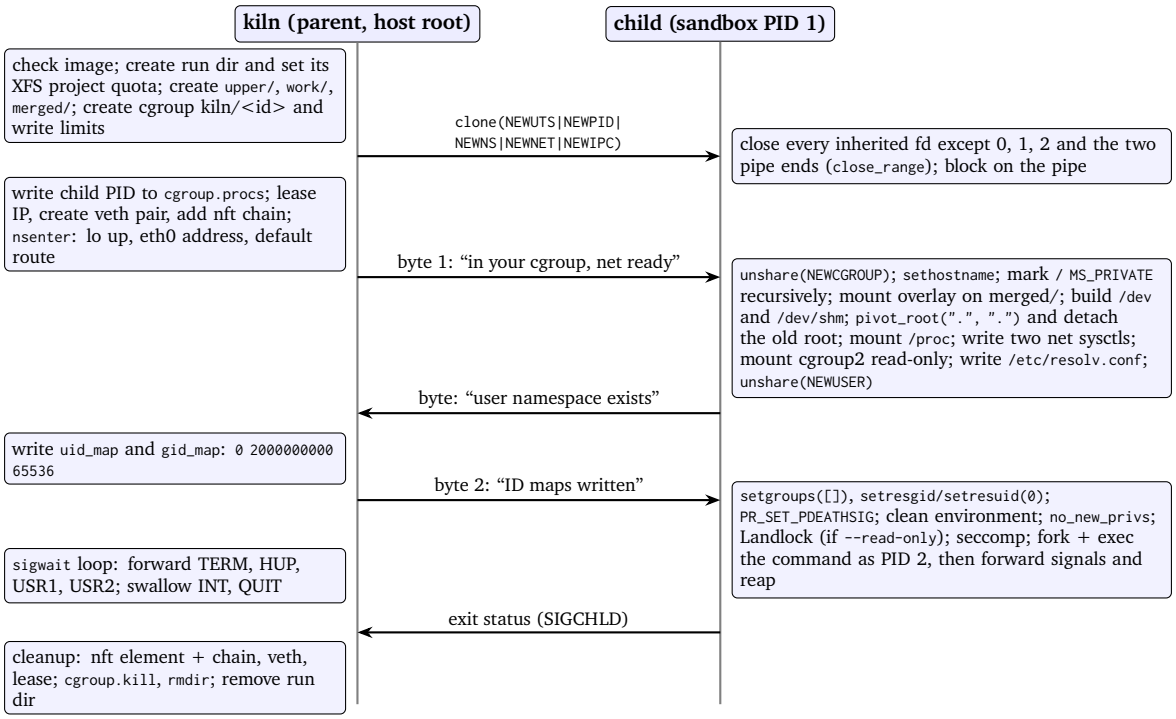


Figure 2: Sandbox startup. Three one-byte messages over two `O_CLOEXEC` pipes order the steps that must happen outside the sandbox (cgroup placement, network plumbing, ID maps) relative to those that must happen inside it. If either side dies, the other reads EOF instead of hanging.

Table 1: Limits, their defaults, and the kernel interface behind each.

Flag	Default	Kernel interface	At the limit
`--memory`	512M	`memory.max`; `memory.swap.max`=0; `memory.oom.group`=1	reclaim, then the OOM killer kills every process in the sandbox
`--cpus`	1	`cpu.max` = "q 100000", q = cores × 100 000 µs	throttled for the rest of each 100 ms period
`--pids`	128	`pids.max`	`fork`/`clone` fail; counted in `pids.events`
`--disk`	1G	XFS project quota: block hard limit, plus one inode per 16 KiB (min. 128)	writes fail with `ENOSPC`
(all runs)	—	parent `io.weight` "default 50" with iocost	proportional share, only under contention
(all runs)	—	tmpfs `size=`: 64 KiB `/dev`, 64 MiB `/dev/shm`	`ENOSPC`; shm pages count toward `memory.max`

everything `MS_REC|MS_PRIVATE`. During a run, `findmnt -t overlay` on the host is empty.

A minimal /dev. Instead of the host's `/dev` (which includes disks), the sandbox gets a 64 KiB tmpfs holding bind mounts of six devices (`null`, `zero`, `full`, `random`, `urandom`, `tty`), the usual `fd` and `std*` symlinks, and a `/dev/shm` tmpfs whose pages are charged to the sandbox's memory limit. A fresh `/proc` (mounted `nosuid,nodev,noexec`) shows only the sandbox's PID namespace.

5 Resource limits with cgroups v2

cgroup v2 is a single hierarchy of directories; each directory is a group of processes, and controller files inside it set limits and report usage. Two rules shape kiln's layout (Figure 4). Controllers are enabled top-down: a cgroup can only use `memory`, `cpu`, `pids` or `io` if its parent lists them in `cgroup.subtree_control`. And a cgroup that delegates controllers to children cannot hold processes itself ("no internal processes"). So sandboxes live *under* `/sys/fs/cgroup/kiln/`, which itself stays empty.

Table 1 lists every limit. The design decisions behind them:

- **Kill the sandbox, not a process.** `memory.oom.group` makes an OOM kill take out the whole group. A half-killed tool call (a shell whose child died) is harder to reason about than one that failed outright. Swap is disabled for the group (when a memory limit is set) so a memory hog hits its limit instead of degrading into swap.

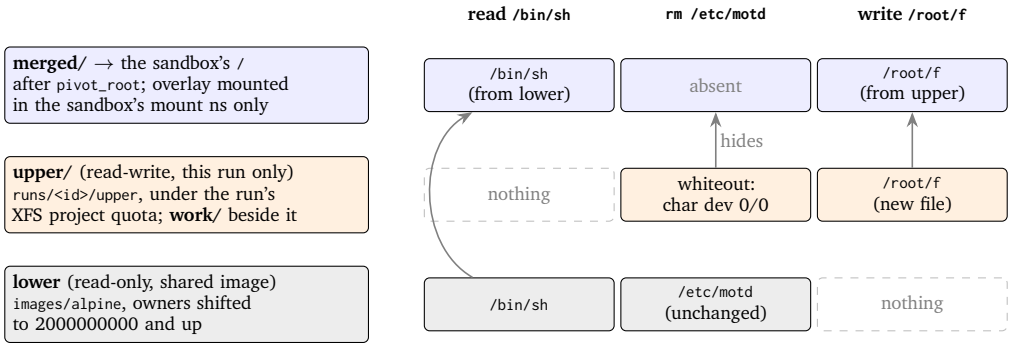


Figure 3: The overlay root. Reads fall through to the shared image; writes and deletions land only in the run's upper layer, so `upper/` is an exact record of what the sandbox changed. On top of `merged/`, the sandbox mounts its own `/proc`, a minimal `/dev` tmpfs, a 64 MiB `/dev/shm` and a read-only cgroup2 at `/sys/fs/cgroup`.

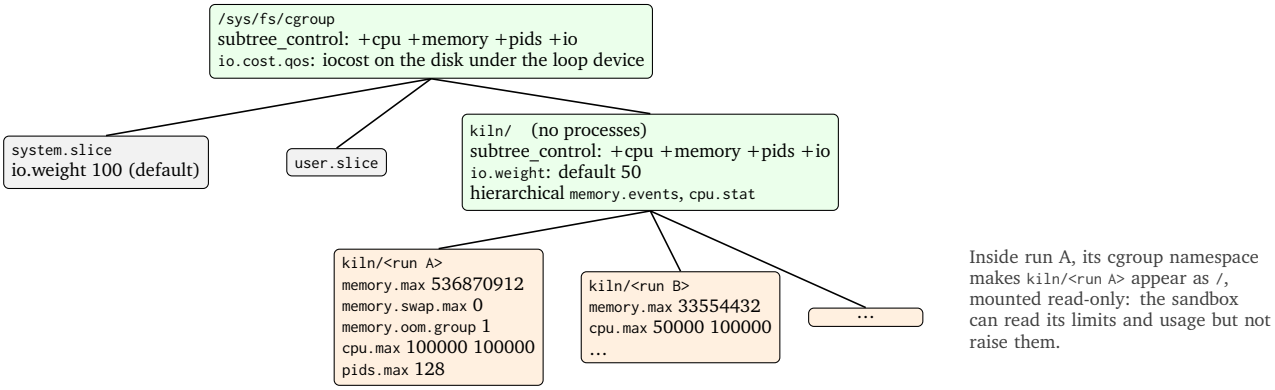


Figure 4: cgroup hierarchy. Limits are set per run; the parent sets the sandboxes' collective I/O share and provides counters that survive individual runs.

- **CPU is a quota per period**, not a share: `--cpus 0.25` writes `25000 100000`, so a 2 s busy loop gets 500 ms of CPU and is throttled once in each of 20 periods (`nr_throttled` in `cpu.stat`).
- **Page cache is memory.** Writing 100 MB to disk showed a memory peak of about 100 MiB, because file pages are charged to the writer's cgroup. They are reclaimable, so they cause writeback rather than OOM kills, but they count toward `memory.max`, which matters for §12.
- **Teardown is atomic.** `cgroup.kill` SIGKILLs every process in the group in one write, including anything that escaped the command's process tree; the directory is then removed once empty.

6 kiln-init: PID 1 and signals

The kernel treats a PID namespace's init specially: signals sent to it from inside its namespace are dropped unless it installed a handler, and orphaned processes are re-parented to it. A normal program as PID 1 therefore ignores `kill` and `timeout` and never reaps orphans. Before kiln-init, `kiln run -- timeout 2 sh -c 'while :; do :; done'` never ended: busybox `timeout` signalled from inside the namespace and PID 1 ignored it.

So PID 1 is kiln itself (the same binary, already running in the child), playing the role `tini` plays for Docker. It blocks all signals, forks the command as PID 2 (which restores an empty mask and `exec`s), and then loops on `sigwait`: on SIGCHLD it reaps *every* exited child (several exits can coalesce into one signal), and SIGTERM, SIGHUP, SIGUSR1 and SIGUSR2 are forwarded to the command. When the command exits, init returns its status (128 + signal number if it was killed, 127 if `exec` failed) and exits; the kernel then kills whatever is left in the PID namespace.

Waiting for signals rather than handling them. Both kiln-init and the supervisor block signals with `sigprocmask` and consume them synchronously with `sigwait` in the same loop that reaps children. There are no asynchronous handlers, so there are no async-signal-safety rules to get wrong. The supervisor therefore survives Ctrl-C and `kill`, passes them on, and still cleans up.

Not forwarding SIGINT. The terminal sends SIGINT, SIGQUIT and SIGWINCH to its whole foreground process group, and the sandboxed command is in that group, so it already received them. Forwarding would deliver them twice. Ctrl-C ends a run with 130 and full cleanup.

7 Disk quotas with XFS project quotas

Without a disk limit, one sandbox can fill the host filesystem. Classic quotas limit what a UID owns, which does not fit (all sandboxes map to the same host IDs, §8). XFS *project* quotas instead limit whatever is tagged with a 32-bit project ID, and the `PROJINHERIT` flag on a directory makes everything created beneath it inherit the tag. This is how Docker's overlay2 driver limits container size.

Storage. `/var/lib/kiln/runs` is an XFS filesystem inside a sparse file (20 GB by default), mounted through a loop device with `prjquota`, so no spare partition is needed. Per run, kiln tags the still-empty run directory with `FS_IOC_FSSETXATTR` (project ID plus `PROJINHERIT`) and sets the limits with `quotactl_fd` (`Q_XSETQLIM`), both as raw `ioctl`/syscall calls against `#[repr(C)]` structs whose sizes are asserted at compile time. `upper/` and `work/` are created afterwards, so they and everything the sandbox writes are charged to the run.

Design decisions. (1) The project ID is the run directory's inode number: unique on the filesystem for as long as the directory exists, so no ID allocator is needed. (2) Because inode numbers are reused, kiln *always* writes the limit, even "unlimited" (0); otherwise a new run could inherit a dead run's limit. (3) Blocks alone are not enough: empty files cost almost no blocks, so a small block quota could still exhaust the filesystem's inodes. Each project also gets an inode limit of one per 16 KiB of quota (at least 128). (4) A full quota surfaces to the program as `ENOSPC` ("No space left on device"), not `EDQUOT`, which is the honest description from inside.

8 User namespaces and ID mapping

Without a user namespace, root in the sandbox is root on the host, and every other wall becomes the last one. With it, sandbox UIDs and GIDs 0–65535 map to host IDs 2 000 000 000–2 000 065 535, a range no real user owns (well above the `/etc/subuid` range given to the VM's user). An escaped process lands on the host as UID 2 000 000 000, which owns nothing.

Only an outsider can write the maps. A fresh user namespace has no mapping, so its process appears as the overflow ID (`nobody`). The parent, which holds `CAP_SETUID`/`CAP_SETGID` over the parent namespace, writes `0 2000000000 65536` to the child's `uid_map` and `gid_map`; the child then drops supplementary groups and switches to UID/GID 0. That is the second pipe handshake in Figure 2.

Capabilities that apply to nothing. Inside, root has a full capability set (`CapEff: 000001ffffffffff`), but since the mount, UTS, network, IPC, PID and cgroup namespaces are owned by the host's user namespace, `hostname`, `mount`, `umount` and `ip link set` all fail with `EPERM`.

Image files must be shifted. Host-root-owned image files would appear as `nobody` inside, and sandbox root could not modify them. The fetch script therefore adds 2 000 000 000 to every file's owner and group, restoring setuid bits that `chown` clears (Docker's userns-remap approach), and kiln refuses to start if the image root is not owned by the base ID. The upper layer is chowned to the same ID so the sandbox's `/` belongs to sandbox root. The known cost is that all sandboxes share one ID range (§15).

Two sysctls make root-in-a-userns feel normal. Binding a port below 1024 requires `CAP_NET_BIND_SERVICE` in the user namespace that owns the network namespace, which sandbox root is not. Before creating the user namespace, the child sets `ip_unprivileged_port_start=0` and `ping_group_range` through `/proc/sys/net`, which is per network namespace. Docker sets the same two.

9 Syscall filtering and Landlock

Namespaces control what a process can *see*; the kernel is still shared, and every system call is attack surface. Two final restrictions are applied just before the command starts, both inherited by every descendant and irrevocable.

no_new_privs comes first. It guarantees that `exec` can never grant more privilege (setuid bits stop working), which is the precondition for an unprivileged process to install a seccomp filter or Landlock ruleset: otherwise a filtered process could exec a setuid binary that runs with the filter but with more privilege.

seccomp-bpf. A seccomp filter is a small classic-BPF program the kernel runs on every syscall, over a struct holding the architecture, syscall number and arguments. kiln's is hand-assembled. It first checks the architecture, since syscall numbers differ per ABI, and kills the process on a mismatch (and, on x86-64, on x32 syscalls). It then denies, with `EPERM`, about 40 syscalls in a few families: loading kernel modules and kexec; large, historically bug-prone subsystems (`bpf`, `perf_event_open`, `userfaultfd`, `io_uring`); the kernel keyring, which is not namespaced; open-by-handle, which bypasses path checks; `setns` and the whole mount API; and host-wide state (clocks, swap, accounting, kernel log, quotas). Finally it denies any `clone` or `unshare` whose flags include a `CLONE_NEW*` bit. That

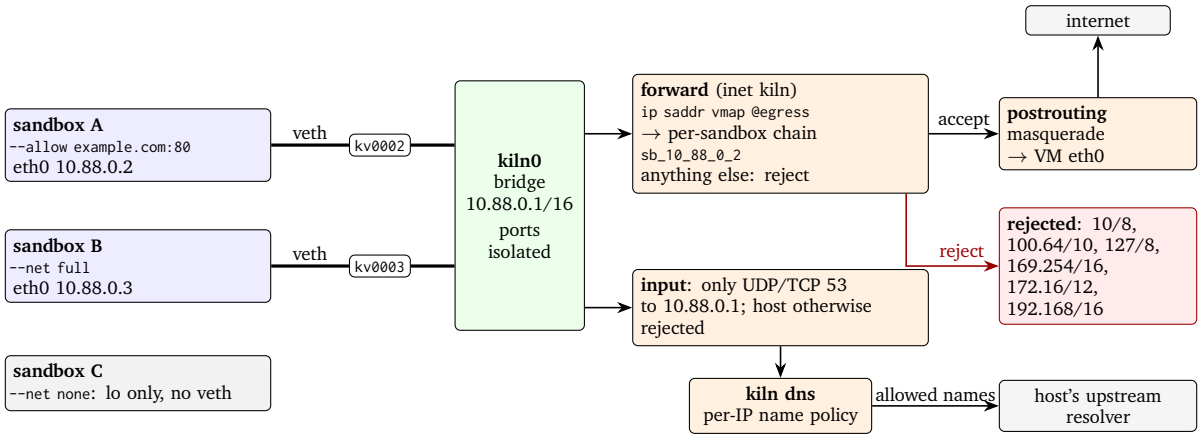


Figure 5: Network topology. Each networked sandbox has a veth pair whose host end is an isolated port on `kiln0`. All policy is in one nftables table: a verdict map sends each sandbox's packets to its own chain, the input chain admits only DNS to the forwarder, and NAT masquerades the rest out of the VM.

matters because sandbox root holds every capability *inside* its user namespace: a new network or mount namespace it created would be its own, exposing root-only kernel code. `clone3` passes its flags in memory the filter cannot read, so it returns `ENOSYS`, which makes libc fall back to `clone` (the approach Docker uses). The filter is installed with `SECCOMP_FILTER_FLAG_TSYNC`, a no-op for today's single-threaded child but correct for a future multi-threaded one.

Decision: a denylist. Docker's default profile is an allowlist of roughly 350 syscalls, which is stricter. kiln chose a short denylist because it can be read and reasoned about in one sitting; the cost is that new kernel syscalls are allowed by default.

Landlock (with `--read-only`) restricts which paths may be written, and unlike file permissions it binds root (`CAP_DAC_OVERRIDE` does not apply). A ruleset declares the access rights it *handles*; any handled right is denied unless a rule grants it beneath some directory. kiln handles only write-type rights (writing, truncating, creating, removing and renaming across directories), so reading and executing stay allowed everywhere, and grants them beneath `/tmp`, `/root`, `/workspace`, `/dev/shm` and any `--writable` paths; existing `/dev` files stay writable but no new devices can be created. kiln requires Landlock ABI 3 (Linux 6.2), the first that covers truncation; on older kernels `--read-only` would silently allow files to be emptied. Landlock does not cover metadata, so `chmod` on a file the sandbox owns still works.

10 Networking: bridge, veth pairs and nftables

Every sandbox gets its own network namespace. With `--net none` (the default) it contains only loopback. With `--net full` or `--allow`, the supervisor connects it to the host (Figure 5).

Plumbing from outside. The network namespace is created at `clone` time, before the user namespace, so sandbox root cannot change its address, routes or firewall. The supervisor configures it before releasing the child: it leases an address, creates a veth pair whose peer is created directly inside the sandbox's namespace as `eth0`, attaches the host end to `kiln0` as an isolated port, installs the sandbox's nftables chain, and then runs `ip` inside the namespace via `nsenter` to bring up `lo`, assign the address and add a default route via 10.88.0.1. The bridge and the base ruleset are created once per boot (checked on every run under an `flock`), and `ip_forward` is enabled.

Leases without a lock. A lease is a file in `/run/kiln/net/leases` created with `O_EXCL`, which is atomic, so concurrent runs cannot get the same address. Because `/run` is a tmpfs, leases vanish on reboot together with the bridge and rules they refer to.

Policy is one verdict map. The base table's forward chain begins:

```
iifname "kiln0" ip saddr vmap @egress   # jump to this sandbox's chain
iifname "kiln0" counter reject with icmpx admin-prohibited
oifname "kiln0" ct state established,related accept
oifname "kiln0" counter drop
```

Adding a sandbox is one map element plus one chain; removing it deletes both, with no rule handles to track. For `--net full` the chain rejects the private and link-local set (the VM's LAN, the Mac behind it, cloud metadata at

169.254.169.254) and accepts the rest. For `--allow HOST[:PORT]` it accepts only the listed addresses (hostnames are resolved once, at start) and rejects everything else. The sandbox's source address is trustworthy because sandbox root cannot change it. Rules end in `reject` rather than `drop`, so a blocked connection fails in milliseconds instead of hanging until a timeout, which matters for an agent waiting on a tool call.

Isolation between sandboxes. Frames between two ports of one bridge are switched, not routed, so the inet forward hook never sees them. kiln sets the bridge port flag `isolated on` instead, which stops isolated ports from talking to each other at layer 2. The input chain keeps sandboxes off the host's own services (`sshd`, and later kilnd's API).

11 The DNS forwarder

Name resolution was the last way out. Originally, networked sandboxes could reach the VM's resolver directly, which on Lima is a port on the Mac, and DNS can carry data in query names (`<secret>.attacker.example`). Now each networked sandbox's `/etc/resolv.conf` (written as real root, so read-only for sandbox root) points at `kiln dns` on 10.88.0.1:53 (UDP and TCP), the only resolver the input chain lets it reach.

Policy by source address. For each query, the forwarder reads the policy file named after the sender's IP in `/run/kiln/net/dns/`: `*` for `--net full`, otherwise the `--allow` hostnames, one per line, each also matching its subdomains. Allowed queries are forwarded to the host's upstream resolver (read from systemd-resolved's `resolv.conf`, since 127.0.0.53 only exists in the host's namespace); others get `REFUSED`; a missing file means an unknown sender and is refused. With `--allow`, this closes DNS as an exfiltration channel: an attacker's domain is never resolved.

Minimal parsing. The forwarder parses only what it needs: the 12-byte header (exactly one question), then length-prefixed labels, then QTYPE and QCLASS. Compression pointers, which a query has no use for, are rejected. Malformed queries are dropped. Upstream replies are accepted only if the transaction ID matches; an upstream failure becomes `SERVFAIL` so the client does not wait for its own timeout.

Bounded by design. Sandboxes can send it unlimited traffic, so the forwarder must not turn that into unlimited host resources: 16 worker threads consume a queue of 256 jobs, and when the queue is full new UDP datagrams and TCP connections are dropped, which DNS clients handle by retrying. The systemd unit caps tasks and memory and sets `NoNewPrivileges`. Every query is logged to the journal with control characters escaped (a query name is attacker-chosen bytes) and counted for Prometheus by outcome and type. The worker bound, the log escaping and the unit's task cap came from the second review (§14).

12 Storage and I/O accounting

An OOM kill caused by someone else's cache. A sandbox writing 1.5 GB with `O_DIRECT` under a 512 MB limit was OOM-killed. Its own writes bypassed the page cache, but the loop driver, by default, writes the XFS image's backing file *through* the page cache, and those dirty pages were charged to the sandbox that caused the I/O. It held nothing reclaimable of its own. Setting direct I/O on the loop device (`LOOP_SET_DIRECT_IO`) removed the double caching: the same test peaked at 2.4 MiB, and writes got faster. The setting does not survive reboot, so kiln checks it on every run.

Proportional I/O. `io.weight` only does something under a proportional I/O controller. The VM's disk uses `mq-deadline` and the kernel has no BFQ, so kiln enables iocost (`io.cost.qos`) on the real disk under the loop device and gives all sandboxes together half the weight of a host service. With loop direct I/O, a sandbox's writes reach the real disk tagged with its cgroup and appear in its `io.stat`. *Honest result:* on this VM, weights do not change throughput yet. iocost only enforces weights when its model judges the device saturated, and the Mac's SSD behind virtio never reaches that under the default model; tuning `io.cost.model` is the next step. The accounting works; enforcement is unproven.

13 Metrics: Prometheus and Grafana

A stateless exporter. `kiln metrics` keeps no state. Each scrape reads the kernel's own counters: `memory.current`, `memory.stat`, `cpu.stat`, `pids.current`, `io.stat` and the `*.pressure` (PSI) files of each sandbox's cgroup; the XFS project quota of its run directory; the host end of its veth in `/sys/class/net`; and `nft list table inet kiln`. It writes the Prometheus text format by hand (`# HELP`, `# TYPE`, then `name{labels} value`; counters end in `_total`). Prometheus scrapes it every 5 s, because most sandboxes live for seconds and the default 1 min interval would miss them.

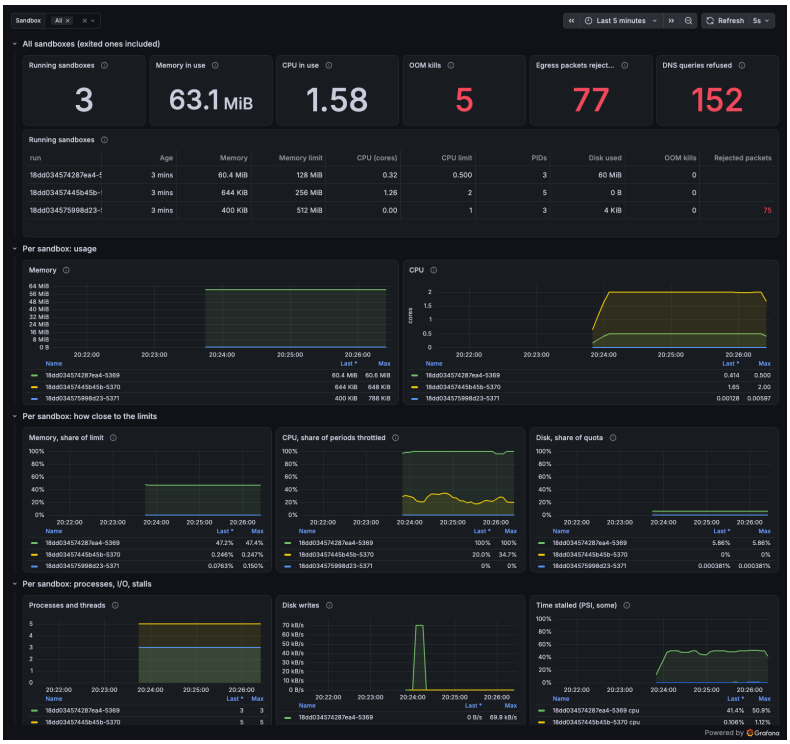


Figure 6: The kiln Grafana dashboard during `scripts/metrics-demo.sh`: one sandbox under `--cpus 0.5` (throttled in 100% of periods, about 50% CPU pressure), one using two cores, one hitting its allowlist (rejected packets, refused DNS queries), and OOM kills counted from the parent cgroup although those sandboxes exited between scrapes.

Grafana is provisioned with a kiln dashboard; anonymous visitors can view it, and the administrator password is generated at install time instead of the default.

Short runs fall between scrapes. A sandbox OOM-killed half a second after starting never appears in a 5 s scrape; its cgroup is gone. The fix is to read the *parent*: `kiln/memory.events` and `kiln/cpu.stat` are hierarchical and keep counting after children are removed, so `kiln_all_oom_kills_total` and `kiln_all_cpu_seconds_total` include every run since boot. Not everything aggregates: the parent's `nr_throttled` counts only throttling by the parent's own limit. The firewall has the same problem, since a sandbox's chain and its counters are deleted with it; each reject therefore also bumps a named counter, `egress_rejected`, that lives as long as the table.

Reading the numbers correctly. A veth pair is a mirror: what the sandbox sends is *received* by the host end, so the exporter swaps rx and tx. An OOM kill counts processes, not sandboxes: with `memory.oom.group`, `oom_kill` rises by one per process killed. Throttling is reported as the share of 100 ms periods in which the quota ran out; a busy loop under `--cpus 0.5` sits at 100%, and `cpu.pressure` shows the same as about 50% of wall time stalled. The exporter's HTTP server, like the DNS forwarder, uses a small fixed worker pool and a bounded queue.

14 Independent review: what it found and taught

kiln was reviewed independently twice. After networking landed, four reviewers attacked it with the goal of breaking isolation. After the metrics layer, a second review by a separate security-focused model ("Daybreak") examined the whole system, including the host-side services. It found five issues; all five were fixed and each fix was verified with a regression script. Table 2 summarizes both rounds by class of issue, not by individual reproduction.

The review also produced simplifications: one nftables table instead of two (NAT works in `inet` tables, and the old second table was never recreated if missing); the veth peer created directly inside the sandbox as `eth0`; the two network sysctls written by the sandbox itself through `/proc/sys/net` instead of a shelled-out `sysctl`; Landlock handling only write rights. Two issues were documented rather than fixed: Landlock does not cover metadata, and DNS to the host's resolver was reachable. The second was later closed by the forwarder. Separately, testing found the `O_DIRECT` OOM of §12. The meta-lesson of the first round: each layer was correct in isolation; the bugs were at the seams, in what one mechanism assumed another had already handled. The second round's lesson:

Table 2: Classes of issues found, the fix, and the design lesson.

Class	Finding and fix	Lesson
A missing namespace	No IPC namespace: sandboxes shared System V IPC objects, which could also outlive the run and its memory limit. Fix: `CLONE_NEWIPC`.	Enumerate every kernel-global resource. User namespaces do not separate sandboxes that share one ID range.
Inherited handles	A file descriptor open in kiln's caller was readable and writable inside a `--read-only` sandbox. Fix: close all but stdio and the pipes first.	Path checks do not revoke handles. Same class as runc's CVE-2024-21626.
State that outlives its owner	conntrack entries and reused IPs let a new sandbox continue a flow only the previous one was allowed; an idempotent `add chain` kept a crashed run's rules. Fix: policy on every outbound packet; flush after add.	Anything keyed by a reusable identifier (IP, inode number) must be reset, not assumed fresh. The quota code follows the same rule.
Input at a trust boundary	`--image` accepted `../..`; size flags could overflow. Fix: plain names only; checked multiplication.	Validate CLI input as if it came from an agent, because eventually it will.
Lifecycle edge cases	Fix: seccomp `TSYNC`; `PR_SET_PDEATHSIG` set *after* the credential change that clears it; Landlock ABI 3 required so truncation is covered.	Kernel features have ordering and version preconditions; failing loudly beats degrading silently.
Second review (security-focused model, after the metrics layer)		
Unbounded host-side work	The DNS forwarder started a thread per query, so sandboxes could make it create threads without limit; the metrics server handled one connection at a time, so one slow client stalled every scrape. Fix: fixed worker pools with bounded queues that drop on overflow; `TasksMax` on the unit.	Anything a sandbox can talk to is part of the isolation boundary. Its resource use must be bounded independently of what sandboxes do.
A limit on one dimension only	The disk quota capped blocks but not inodes; empty files cost almost no blocks, so one sandbox could exhaust the filesystem's inodes. Fix: an inode limit of one per 16 KiB of quota (minimum 128).	Limit every dimension of a resource: bytes *and* count, as `memory.max` and `pids.max` do for memory and processes.
Untrusted bytes in a trusted channel	DNS query names (arbitrary bytes chosen by the sandbox) were written to the journal unescaped. Fix: escape control characters.	Logs are an output that operators trust; escape attacker-controlled data before it reaches them.
Insecure defaults	Grafana kept its default administrator password. Fix: generate one at install time into a root-only file, and reset it on existing installs.	The observability stack ships as part of the system; its defaults are the system's defaults.

isolating sandboxes moves the attack surface onto the shared services that serve them.

15 Known limitations and future work

Limitations today.

- *No reconcile.* If the supervisor is killed, `PR_SET_PDEATHSIG` kills the sandbox, but its cgroup, run directory and lease leak; after a host reboot, run directories remain.
- *One shared ID range.* All sandboxes map to the same host IDs, so they are not separated from each other by the user namespace. Idmapped mounts (`mount_setattr` with `MOUNT_ATTR_IDMAP`) would allow a range per sandbox without copying the image.
- *Allowlist drift.* `--allow` hostnames are resolved once at start; CDN addresses change. The forwarder also allows subdomains of an allowed name whose addresses the firewall does not. Next step: the forwarder adds the addresses it resolves to the sandbox's nftables set.
- *DNS tunnelling under* `--net full` is logged but allowed.
- *Kernel surface.* The seccomp denylist is weaker than an allowlist, and all sandboxes share the host kernel. IPv4 only.
- *I/O weights* are wired and accounted but not enforced on this VM.

Future work. The build order continues with: (1) **Persistent sandboxes.** Agents need state between tool calls (installed packages, files), so a sandbox will live for an agent session and run many commands via `setns`, like `docker`

`exec`. A daemon, `kilnd`, will own sandbox state as a state machine with run records in SQLite, reconcile on startup by scanning `/sys/fs/cgroup/kiln/` and `/var/lib/kiln/runs/`, and run one shim process per sandbox so sandboxes survive a daemon crash or upgrade (the reason containerd has `containerd-shim`). The isolation backend sits behind a trait so a Firecracker microVM backend can offer the same API with a separate kernel. (2) **The agent harness**: an agent loop with long-term memory and tools (`exec`, `read_file`, `write_file`, `diff`) talking to kilnd over gRPC on a Unix socket, with the overlay upper layer as the audit log and the basis for snapshots, and an egress proxy that logs requests. (3) **Cold start**: measure, then optimize. (4) **A Kubernetes operator**: k3s across several VMs, a Go operator with a `Sandbox` custom resource that places sandboxes on nodes, and kilnd on each node as a DaemonSet.

16 Conclusion

A container is not one feature but a set of independent kernel mechanisms, each closing one door, and most of the engineering is in their ordering and their seams: limits must apply before the first instruction, the user namespace must come last, descriptors must be closed before anything else, and every identifier the kernel reuses must be reset. Building each layer directly against the kernel, measuring its failure mode, and then having others try to break it made those seams concrete.

References

[1] Linux manual pages: *namespaces*(7), *user_namespaces*(7), *pid_namespaces*(7), *cgroup_namespaces*(7), *clone*(2), *pivot_root*(2), *seccomp*(2), *landlock*(7), *quotactl*(2).

[2] T. Heo et al., *Control Group v2*, Linux kernel documentation, `Documentation/admin-guide/cgroup-v2.rst`.

[3] *Overlay Filesystem*, Linux kernel documentation, `Documentation/filesystems/overlayfs.rst`.

[4] M. Salaün, *Landlock: unprivileged access control*, Linux kernel documentation, `Documentation/userspace-api/landlock.rst`.

[5] *xfs_quota*(8) and *XFS project quotas*, xfsprogs documentation.

[6] CVE-2024-21626: runc file descriptor leak ("Leaky Vessels"), GitHub advisory GHSA-xr7r-f8xq-vfvv, January 2024.

[7] Docker documentation: *Seccomp security profiles for Docker*; *Isolate containers with a user namespace* (userns-remap).

[8] *tini*: a tiny but valid init for containers, `github.com/krallin/tini`.

[9] *nftables wiki*: verdict maps, NAT in inet tables, named counters.

[10] Prometheus documentation: *Exposition formats*.

A Common questions and answers

Positioning

Q: How does kiln compare with Docker and runc?

Docker is a stack: a daemon, containerd, a per-container shim, and runc, which does the actual namespace, cgroup and mount setup from an OCI bundle. kiln covers roughly runc's job plus the parts of Docker that matter for sandboxing (image storage with overlayfs, bridge networking with NAT, disk quotas, an egress policy, metrics) in one binary. It has no OCI image or runtime-spec support, no image distribution, and no daemon yet. It exists for depth, not to replace them.

Q: And with gVisor and Firecracker?

All three answer "how much of the host kernel does untrusted code reach?" kiln, like runc, lets the sandbox call the host kernel directly and narrows that with namespaces, seccomp and Landlock. gVisor intercepts syscalls and serves most of them from a user-space kernel, so far less host kernel code is reachable, at a cost in syscall-heavy workloads. Firecracker runs a guest kernel in a KVM microVM: the strongest boundary of the three, at the cost of a VM per sandbox. kiln's planned backend trait is meant to let a microVM backend sit behind the same API.

Q: Why Rust?

A runtime runs as root and parses untrusted input (DNS packets, HTTP requests), so memory safety matters. `unsafe` is confined to small, commented blocks around raw syscalls and `ioctl`s, which makes the dangerous parts easy to audit; `#[repr(C)]` structs have their sizes checked at compile time; and `Drop` gives every kernel object (cgroup, nft chain, lease) cleanup on every exit path. Rust also has no runtime threads, so `clone` and `fork` behave as in C. Go's multi-threaded runtime is why runc does its namespace setup in a small C shim before Go starts.

Q: Why no daemon yet?

A one-shot CLI was enough to learn each kernel layer, and it keeps the state model trivial: everything lives in the kernel and a few directories, so there is nothing to get out of sync. A daemon becomes necessary once sandboxes outlive one command. The current design prepares for it: run IDs name cgroups and run directories, so a daemon can rebuild its view by scanning them.

Mechanisms

Q: How do you guarantee limits apply before the command runs?

The child blocks on a pipe immediately after `clone`. The parent writes its PID to `cgroup.procs` and sets up networking, and only then writes the byte that releases it. `clone3` with `CLONE_INTO_CGROUP` would do this atomically.

Q: Why is the user namespace created last?

Each namespace is owned by the user namespace current at its creation, and capabilities apply only to namespaces one's user namespace owns. Creating it last means the mount, network, UTS, IPC, PID and cgroup namespaces belong to the host, so sandbox root's capabilities do not reach them.

Q: Sandbox root has all capabilities in its user namespace. Can't it just create new namespaces?

That is exactly what the seccomp filter prevents: any `clone` or `unshare` with a `CLONE_NEW*` flag gets `EPERM`, `clone3` gets `ENOSYS`, and `setns` and the mount API are denied.

Q: Why does the command run as PID 2?

PID 1 of a namespace ignores signals from inside it unless it installed a handler, and inherits all orphans. kiln-init forwards termination signals and reaps zombies, so `timeout`, `kill` and long-running shells behave normally.

Q: Why overlayfs instead of copying the image for each run?

Copying costs time and disk per run; an overlay mount is constant-time, every sandbox shares the image's page cache, and the upper layer is an exact diff of the run, which is both an audit log and the basis for snapshots.

Q: Why cgroup v2?

One hierarchy instead of one per controller, and the features kiln relies on exist only there: `memory.oom.group`, `cgroup.kill`, pressure stall information, iocost, and cgroup namespaces that show a sandbox only its own subtree. Ubuntu 24.04 uses v2 by default.

Q: What does an agent see when a sandbox runs out of memory?

`memory.oom.group` kills every process in the sandbox at once, so the run ends with exit code 137 (128 + SIGKILL), and the supervisor reads `oom_kill` from `memory.events` and says so. Writing large files does not cause this: page cache is charged but reclaimable.

Q: Why XFS project quotas rather than limiting by UID?

All sandboxes share one host ID range, so a UID quota could not tell them apart. A project quota limits a directory tree, and `PROJINHERIT` tags everything the sandbox creates. The run directory's inode number serves as a free, collision-free project ID, and each project gets both a block and an inode limit.

Q: Why did a sandbox using O_DIRECT get OOM-killed?

The loop device under the XFS image buffered the backing file in the page cache, and those pages were charged to the sandbox's cgroup. Switching the loop device to direct I/O removed the double caching.

Q: Why reject instead of drop, and why isn't "established" accepted first?

Reject makes blocked connections fail fast, which an agent loop needs. Outbound packets are always checked against the sandbox's current policy because IPs are reused and conntrack entries outlive sandboxes; "established" is trusted only for replies coming back in.

Q: How do you stop data exfiltration through DNS?

Sandboxes can reach only kiln's forwarder. With `--allow`, it resolves only the allowed names and their subdomains and refuses the rest, so an attacker's domain never reaches a resolver. With `--net full`, queries are logged but allowed.

Q: Why is the DNS forwarder a separate service?

It listens on one shared address, 10.88.0.1:53, for every sandbox, and it keeps no per-run state: each run only writes a policy file named after its IP. A single long-lived service under systemd, with its own task and memory caps, is simpler and easier to bound than a forwarder per run.

Operations

Q: How do metrics survive short-lived runs?

A run that ends between two 5 s scrapes takes its cgroup and nftables chain with it. The exporter therefore also reads the parent cgroup, whose `memory.events` and `cpu.stat` are hierarchical and keep counting after children are removed, and a named nftables counter that every reject increments and that lives as long as the table. Totals since boot stay exact even when no scrape ever saw the run.

Q: What happens if the supervisor is killed with SIGKILL?

The sandbox dies too (`PR_SET_PDEATHSIG`). Its cgroup, run directory and IP lease are left behind; the planned daemon will reconcile them on startup. Catchable signals (Ctrl-C, `kill`) are consumed by `sigwait` and lead to a normal cleanup.

Q: How fast does a sandbox start?

It has not been measured yet; cold-start work is on the roadmap after the daemon and the harness. The expected costs are known: creating the cgroup, quota and overlay, and for networked runs several `ip`, `nft` and `nsenter` process launches, which netlink calls would replace.

Q: How would you test it?

Today, each limit was verified by triggering its failure mode on purpose (OOM kill, fork bomb, full disk, rejected packet), `scripts/metrics-demo.sh` exercises the whole stack, and the second review's fixes have a regression script. Next: run that suite in a VM in CI; turn each review finding into a test that asserts the attack now fails; unit-test the seccomp program and size parsing; and fuzz the DNS question parser, the one piece that parses sandbox-controlled bytes on the host.

Q: How would this run on many hosts?

A daemon per node owns local sandbox state and reconciles from the kernel and disk. On k3s, a Go operator with a `Sandbox` custom resource places sandboxes on nodes, with the daemon deployed as a DaemonSet. Node-local state (bridges, leases, quotas) stays node-local; the operator only schedules, using per-node capacity it can read from the same cgroup metrics.

Q: Is this safe for hostile code?

It is defense in depth on a shared kernel: user namespace, seccomp, Landlock, network isolation and resource limits. A kernel bug reachable through an allowed syscall would still be an escape, and all sandboxes share one host ID range. That is why a microVM backend, a seccomp allowlist and per-sandbox ID ranges are on the roadmap.

Q: What would you do differently?

Start from `clone3` with `CLONE_INTO_CGROUP` and pidfds rather than PIDs; use idmapped mounts instead of rewriting image ownership, which would also give each sandbox its own ID range; begin with a seccomp allowlist; talk netlink instead of shelling out to `ip` and `nft`; and write the regression tests alongside each layer rather than after the reviews.